

Line detection

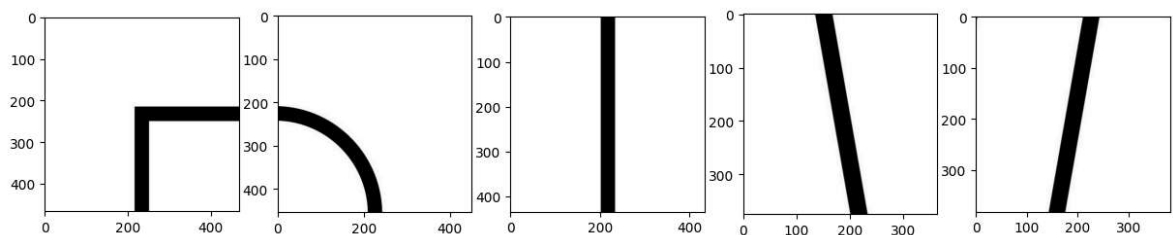
This Jupyter Notebook show how we can detect a line using Hough Transformation

```
In [1]: # Import Libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Load data

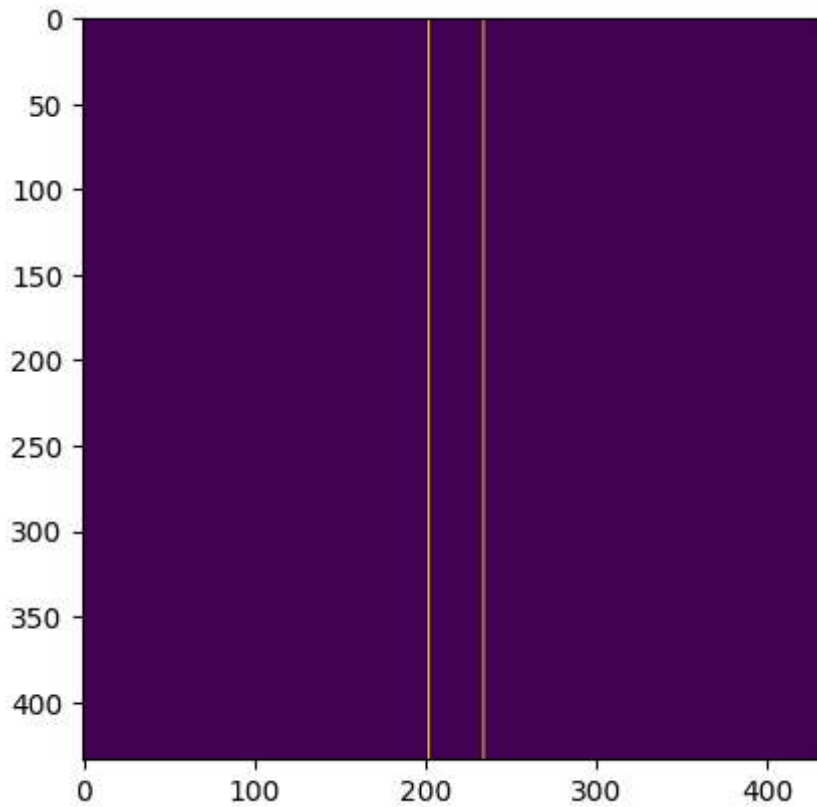
```
In [120]: plt.figure(figsize=(15,6))

plt.subplot(1, 5, 1)
plt.imshow(cv2.imread('datas/ladrinho1.jpg'));
plt.subplot(1, 5, 2)
plt.imshow(cv2.imread('datas/ladrinho2_esquerda.jpg'));
plt.subplot(1, 5, 3)
plt.imshow(cv2.imread('datas/ladrinho3_reto.jpg'));
plt.subplot(1, 5, 4)
plt.imshow(cv2.imread('datas/ladrinho3_esquerda.jpg'));
plt.subplot(1, 5, 5)
plt.imshow(cv2.imread('datas/ladrinho3_direita.jpg'));
```



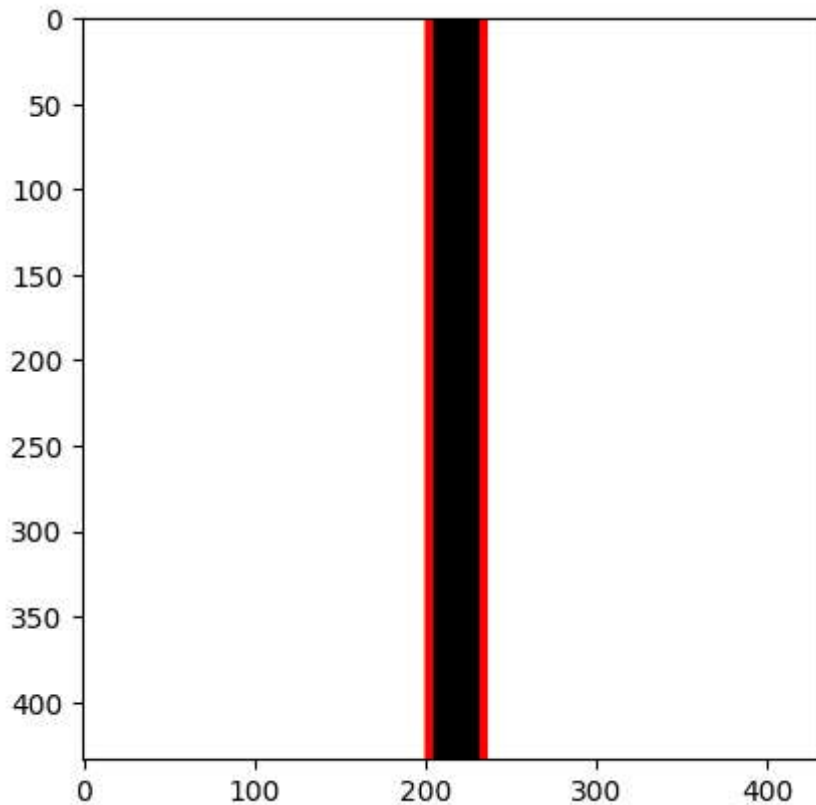
Detecting the line

```
In [11]: img = cv2.imread('datas/ladrinho3_reto.jpg')
img_reserve = img.copy()
# Convert the image to grayscale
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# find the edges in the image
edges = cv2.Canny(img_gray, 50, 200)
plt.imshow(edges);
```



```
In [106... # Detect points that form a line
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 70, minLineLength=15, maxLineGap=25

# Draw lines on the image
for line in lines:
    x1, y1, x2, y2 = line[0]
    cv2.line(img, (x1, y1), (x2, y2), (255, 0, 0), 3)
# Show result
plt.imshow(img);
```



We got detect the line, but we need create a imagination line to make the measure of the car

```
In [107... # determine the size of the line
line_thickness = 32

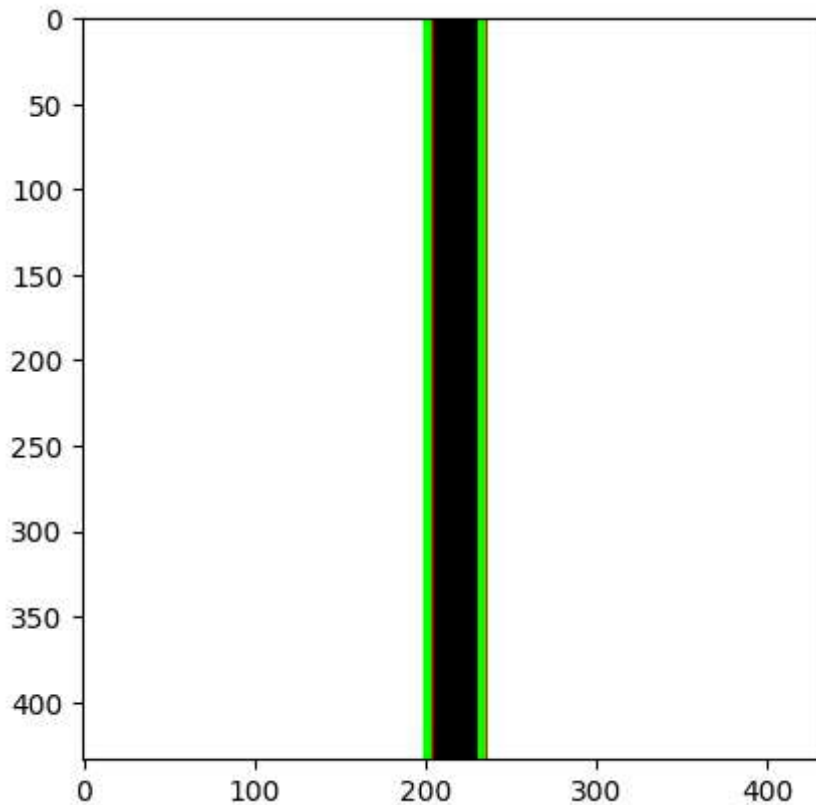
# extract the img size
x, y, _ = img.shape

# determine the position of car flow line in relation of x Label
car_flow_x = int((x - line_thickness) / 2)
start_point = (car_flow_x, 0)
end_point = (car_flow_x, y)

thickness = 3
color=(0, 255, 0)
# drawn the line 1
line_flow_1 = cv2.line(img, start_point, end_point, color, thickness)

start_point = (car_flow_x + line_thickness, 0)
end_point = (car_flow_x + line_thickness, y)
# drawn the line 2
line_flow_2 = cv2.line(img, start_point, end_point, color, thickness)

plt.imshow(img);
```



We got drawn the flow car line

now we've to measure the diff into the path correct(red) and the flow path(green)

```
In [108... # get the correct path
target_line1 = lines[0]
target_line2 = lines[1]

# extract the point x of target_line
start_point_target = target_line1[0][:2][0]
end_point_target = target_line2[0][:2][0]

start_point_target, end_point_target
```

Out[108... (202, 234)

```
In [109... # extract the point x of flow_car
start_point_flow = car_flow_x
end_point_flow = start_point_flow + line_thickness

start_point_flow, end_point_flow
```

Out[109... (201, 233)

```
In [110... # calculate the diff = error
error_left = np.abs(start_point_target - start_point_flow)
error_right = np.abs(end_point_target - end_point_flow)

error_left, error_right
```

Out[110...] (1, 1)

Wow, we got made the measure of target line and car flow. What's happen now?

- We need make a algorithm to adjust the motor speed to fix the carr in target line, but before let's try with another images.

Do you mind. I need do rewrite all code again. A: no, let's make a function!

```
In [2]: def calculate_error(target_line):
        """
        Calculates the error between the target line position and the car flow line

        Args:
            target_line (list): List containing the coordinates of the detected targ
            start_point_flow (int): X-axis starting position of the car flow line.

        Returns:
            int: erro in degrees.
        """
        #print(target_line)
        if isinstance(target_line, tuple):
            target_line = target_line[0]

        # Convert to numpy array e remove the extra dimensions
        contour = np.squeeze(np.array(target_line))

        # avoid erro if has few points
        if contour.shape[0] < 2:
            return None

        # get the extrames points
        x1, y1 = tuple(contour[np.argmin(contour[:, 1])]) # few value of Y (top)
        x2, y2 = tuple(contour[np.argmax(contour[:, 1])]) # bigger value of Y (base)

        # calculate angle em rad
        theta_rad = np.arctan2(y2 - y1, x2 - x1)

        # convert to degree
        theta_deg = np.degrees(theta_rad)

        # to debug
        #print(theta_deg)

        # assure the degrees in [0, 180]
        if theta_deg < 0:
            theta_deg += 180

        # calculate the error
        error = theta_deg - 90

        # return the erro in degree (for Less use of memory)
        return int(error)
```

```
In [3]: def draw_line(img):
        """
        Draws two vertical lines on the image representing the car flow boundaries.
```

```

Args:
    img (numpy.ndarray): The input image on which the lines will be drawn.

Returns:
    tuple: A tuple (left, right) representing the X-coordinates of the left
           boundaries of the car flow line.
"""
# determine the size of the line
line_thickness = 32

# extract the img size
y, x, _ = img.shape

# determine the position of car flow line in relation of x label
car_flow_x = int((x - line_thickness) / 2)
start_point1 = (car_flow_x, 0)
end_point1 = (car_flow_x, y)

thickness = 3
color=(0, 255, 0)
# drawn the line 1
line_flow_1 = cv2.line(img, start_point1, end_point1, color, thickness)

start_point2 = (car_flow_x + line_thickness, 0)
end_point2 = (car_flow_x + line_thickness, y)
# drawn the line 2
line_flow_2 = cv2.line(img, start_point2, end_point2, color, thickness)

# return the position in format (left, right)
return (car_flow_x, car_flow_x+line_thickness)

```

```

In [108... def identify_colour(img):
    """
    Identifies the dominant color in an image based on predefined HSV ranges.

    This function converts the input image to the HSV color space and detects sp
    (red, green, and gray) using predefined hue, saturation, and value ranges. I
    morphological operations to reduce noise, finds contours, and calculates the
    by each detected color. The function returns the color with the largest dete

    Parameters:
    img (numpy.ndarray): The input image in BGR format.

    Returns:
    str or None: The detected color with the largest area ("red", "green", or "g
                  if no significant color region is found
    """
    # convert image in HSV scale
    hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # definy colour ranges
    color_ranges = {
        "red": [(np.array([0, 100, 100]), np.array([10, 255, 255])),
                (np.array([160, 100, 100]), np.array([180, 255, 255]))], # Red
        "green": [(np.array([40, 40, 40]), np.array([90, 255, 255]))], # Green
        "gray": [(np.array([0, 0, 50]), np.array([130, 60, 220]))] # Gray
    }

    kernel = np.ones((3, 3), np.uint8)

```

```

# create a dict to storage the colours
detected_areas = {}

# find colours
for color, ranges in color_ranges.items():
    # create mask colours
    mask = sum(cv2.inRange(hsv_img, lower, upper) for lower, upper in ranges)
    # reduce the noisy
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX)
    detected_areas[color] = sum(cv2.contourArea(c) for c in contours)

    if contours:
        # drawn the contours in black colour
        cv2.drawContours(img, contours, -1, (0,0,0), 2)

if max(detected_areas.values()) > 0:
    # return the colour with most area on the image
    return max(detected_areas, key=detected_areas.get)
# return none if not has colour in the image
return None

```

In [136...

```

def verify_curve(contours, img_widht):
    """
    Verify if has a curve on the image

    Args:
        contours

    Return:
        string: "side of curve"
    """
    vertical = []
    horizontal = []

    for contour in contours:
        x, y, w, h = cv2.boundingRect(contour)

        if h > w:
            vertical.append(x)
        else:
            horizontal.append(x)

    # if not has curve
    if len(vertical) > len(horizontal):
        return 'No curve'

    # if horizontal is bigger
    if len(horizontal) > len(vertical):
        # divide in int number
        mid = len(contours) // 2

        left = [cv2.boundingRect(contour)[0] for contour in contours[:mid]]
        right = [cv2.boundingRect(contour)[0] for contour in contours[mid:]]

        # calcule the avarege
        average_left = np.mean(left) if left else 0
        average_right = np.mean(right) if right else img_widht

```

```

print(average_left, average_right)

# define if is left or right
if average_left > average_right:
    return 'Left'
elif average_left < average_right:
    return 'Right'
# if not identify the side
return 'Green square'

# if not work
return None

```

```

In [137... def detect_line(img):
    """
    Detects lines in the given image using edge detection and Hough Transform,
    draws the detected lines, and calculates the error between the detected line
    and a reference flow line.

    Args:
        img (numpy.ndarray): Input image, which should be in BGR format.

    Returns:
        float: The calculated error between the detected lines and the flow line
    """

    # verify if img exists
    if img is not None:
        # Convert the image to grayscale
        img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Aplly a threshold to detect only darken colours
        _, binary = cv2.threshold(img_gray, 50, 255, cv2.THRESH_BINARY_INV)

        # reduce the noise
        kernel = np.ones((3, 3), np.uint8)
        binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
        binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

        # find the edges on the image
        edges = cv2.Canny(binary, 50, 200)

        # Detect points that form a line
        #lines = cv2.HoughLinesP(edges, 1, np.pi/180, 70, minLineLength=20, maxL
        contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPR

        # send img_widht as img.shape[1]
        print(verify_curve(contours, img.shape[1]))

        # verify if has a symbol on the img
        cor_detected = identify_colour(img)
        if cor_detected:
            print(f'Has the colour {cor_detected} on the image')

        # to debug
        #plt.imshow(binary, cmap='gray')
        #plt.show()

        if contours is not None:
            # drawn the line target

```

```

        cv2.drawContours(img, contours, -1, (255, 0, 0), 2)

        #draw the car flow line
        flow_line = draw_line(img)

        # calculate the error
        erro = calculate_error(contours)

        return erro
    return None

```

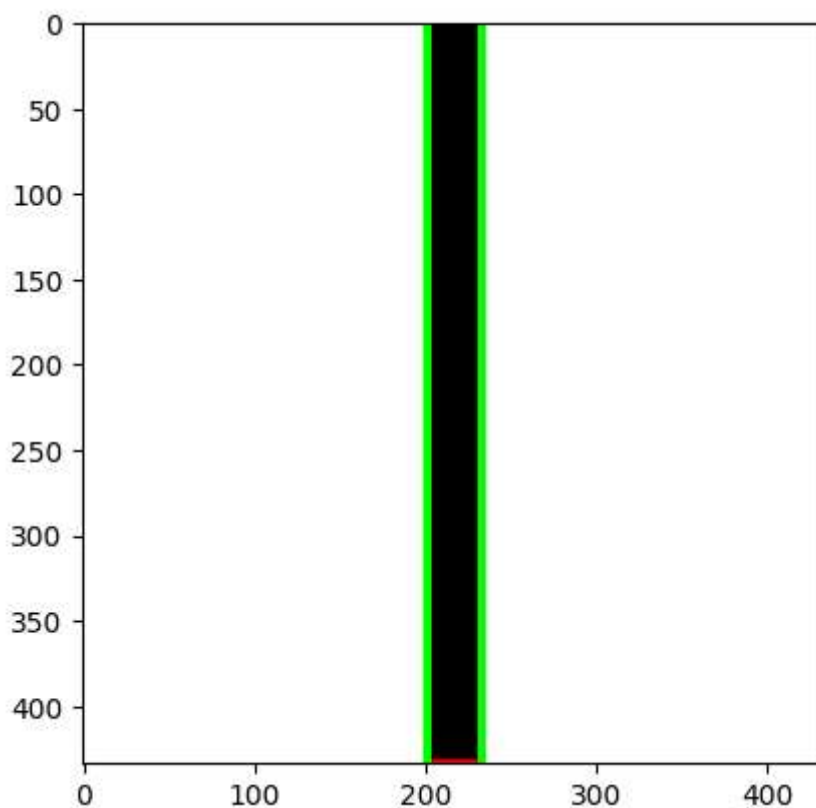
```

In [138... img = cv2.imread('datas/ladrinho3_reto.jpg')
erro = detect_line(img)
plt.imshow(img)
print('The error in degrees:', erro)

```

No curve

The error in degrees: 0



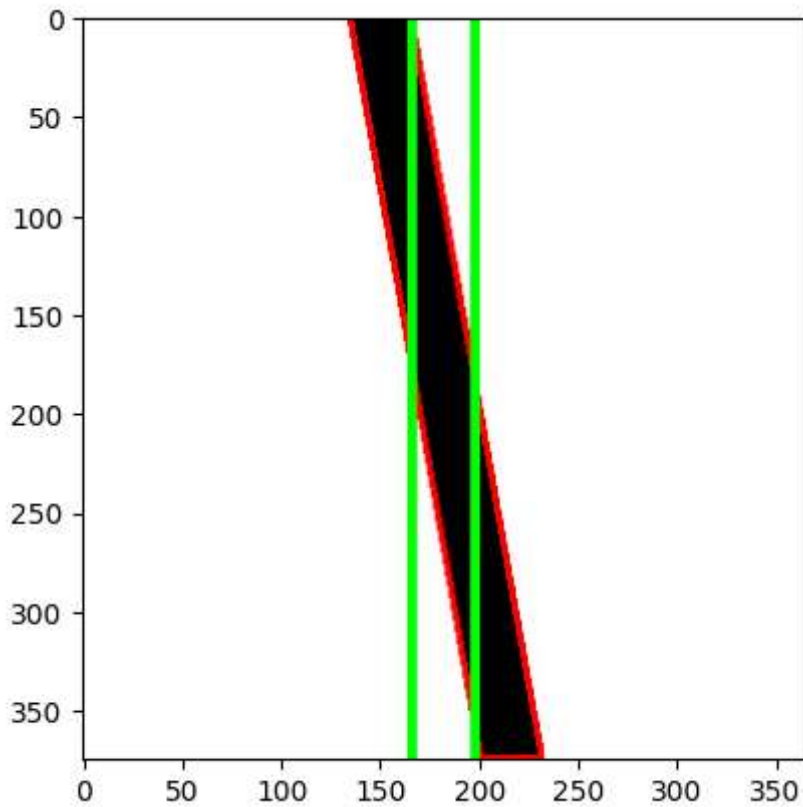
```

In [139... img = cv2.imread('datas/ladrinho3_esquerda.jpg')
erro = detect_line(img)
plt.imshow(img)
print('The error in degrees:', erro)

```

No curve

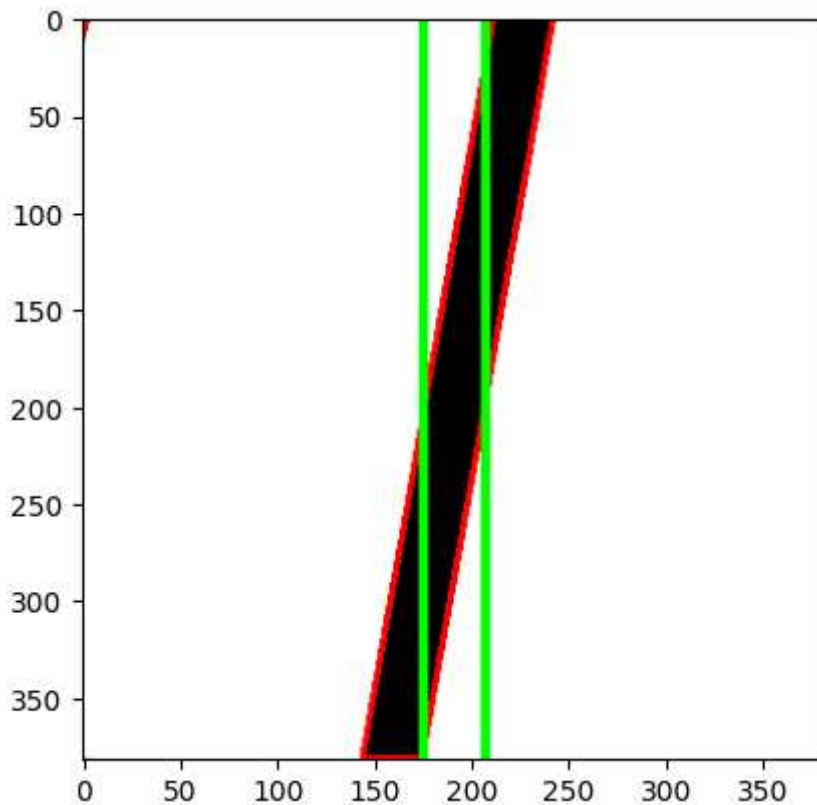
The error in degrees: -10



```
In [140... img = cv2.imread('datas/ladrinho3_direita.jpg')
erro = detect_line(img)
plt.imshow(img)
print('The error in degrees:', erro)
```

No curve

The error in degrees: 9

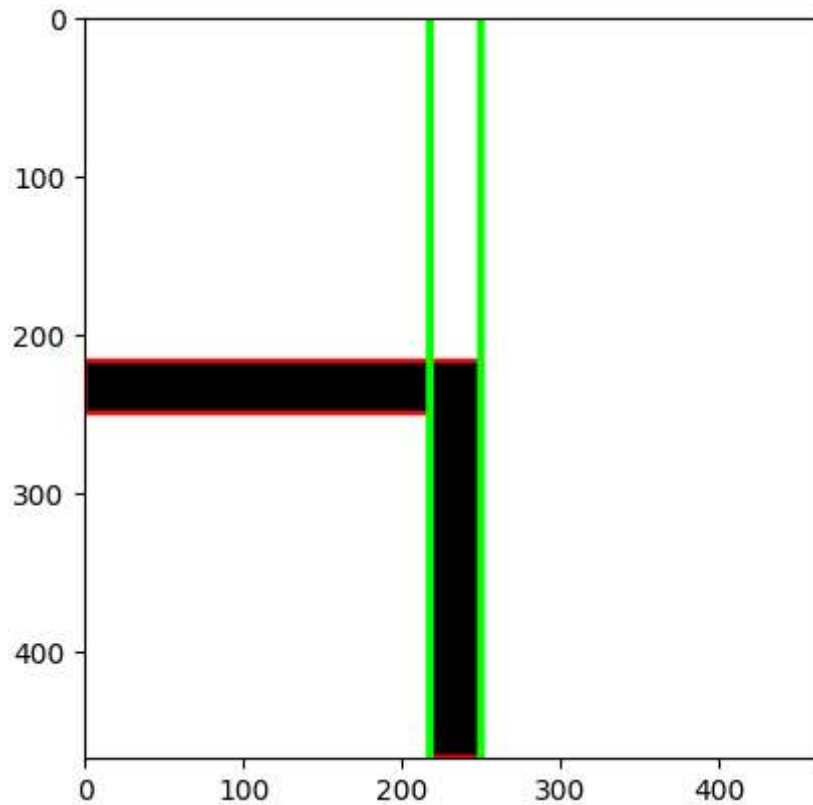


```
In [141... img = cv2.imread('datas/ladrinho1_esquerda.jpg')
erro = detect_line(img)
```

```
plt.imshow(img)  
print('The error in degrees:', erro)
```

No curve

The error in degrees: -40

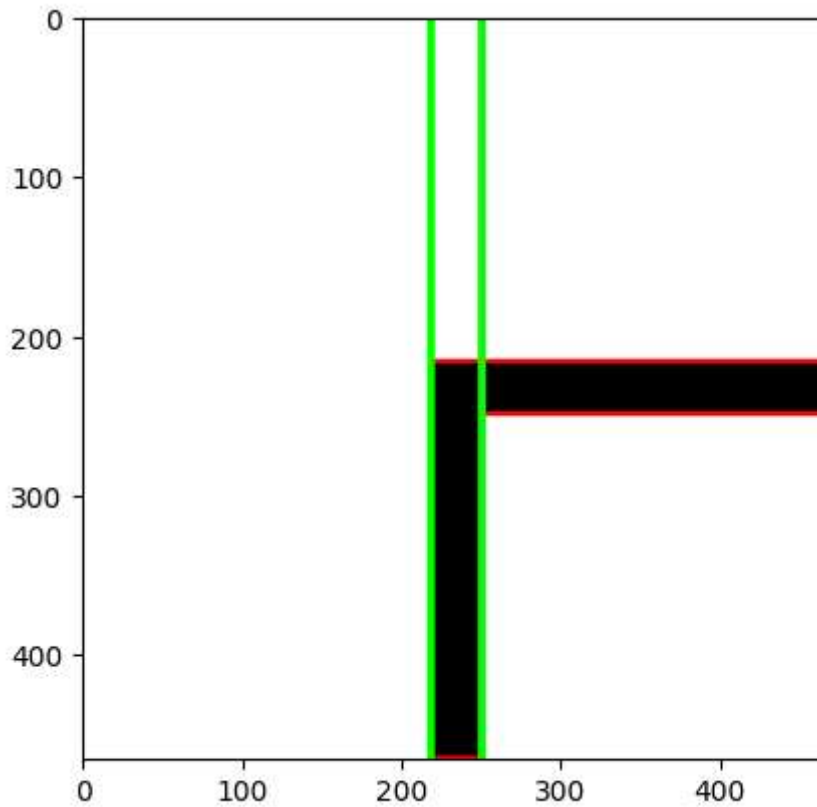


```
In [142... img = cv2.imread('datas/ladrinho1_direita.jpg')  
erro = detect_line(img)  
plt.imshow(img)  
print('The error in degrees:', erro)
```

0 217.0

Right

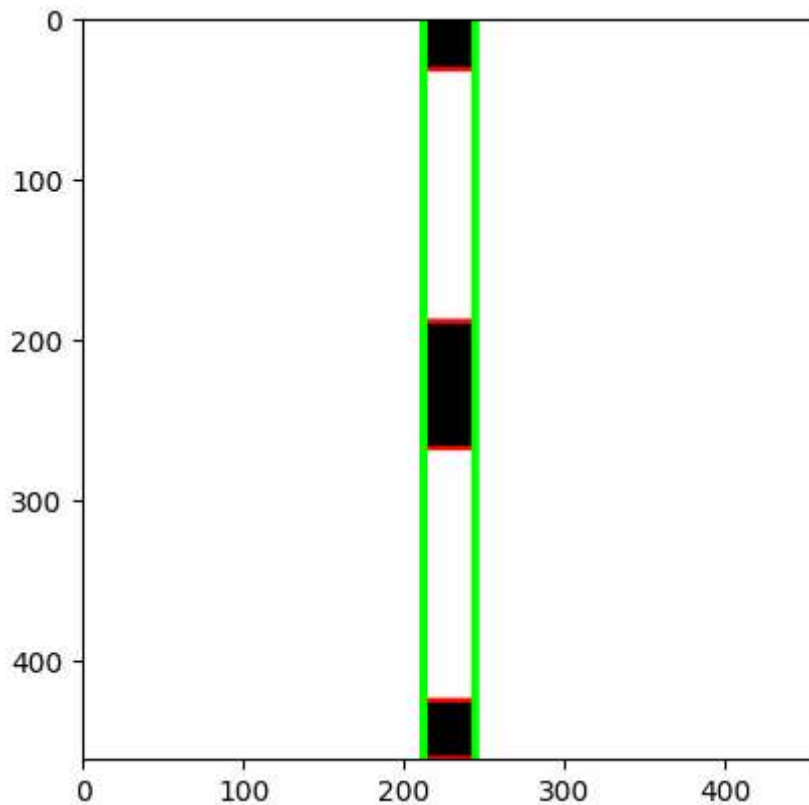
The error in degrees: 0



```
In [143... img = cv2.imread('datas/ladrinho5.jpg')
erro = detect_line(img)
plt.imshow(img)
print('The error in degrees:', erro)
```

No curve

The error in degrees: 0

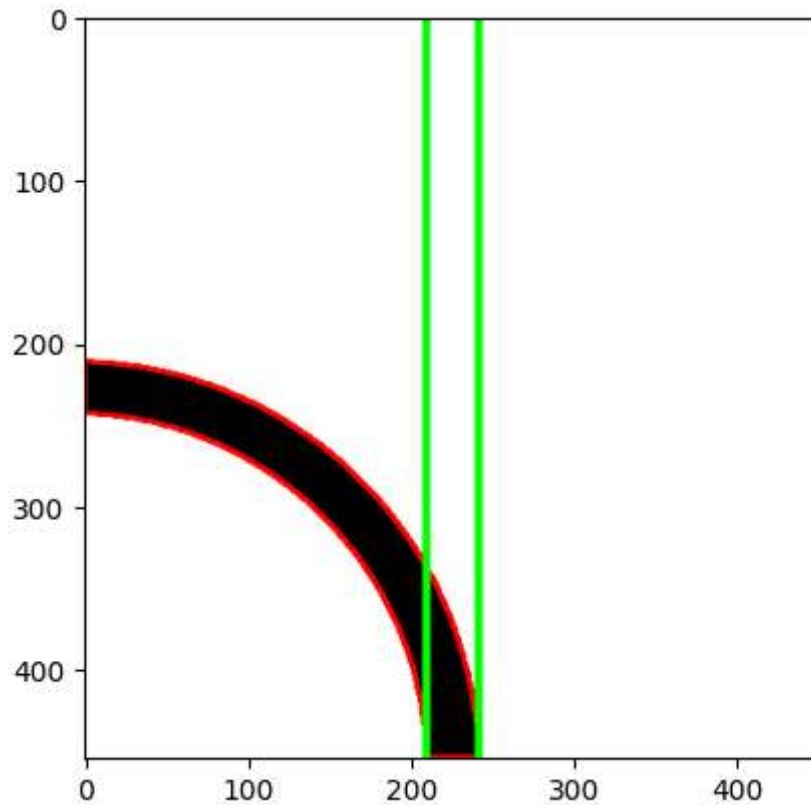


```
In [144... img = cv2.imread('datas/ladrinho2_esquerda.jpg')
erro = detect_line(img)
```

```
plt.imshow(img);  
print('The error in degrees:', erro)
```

No curve

The error in degrees: -40

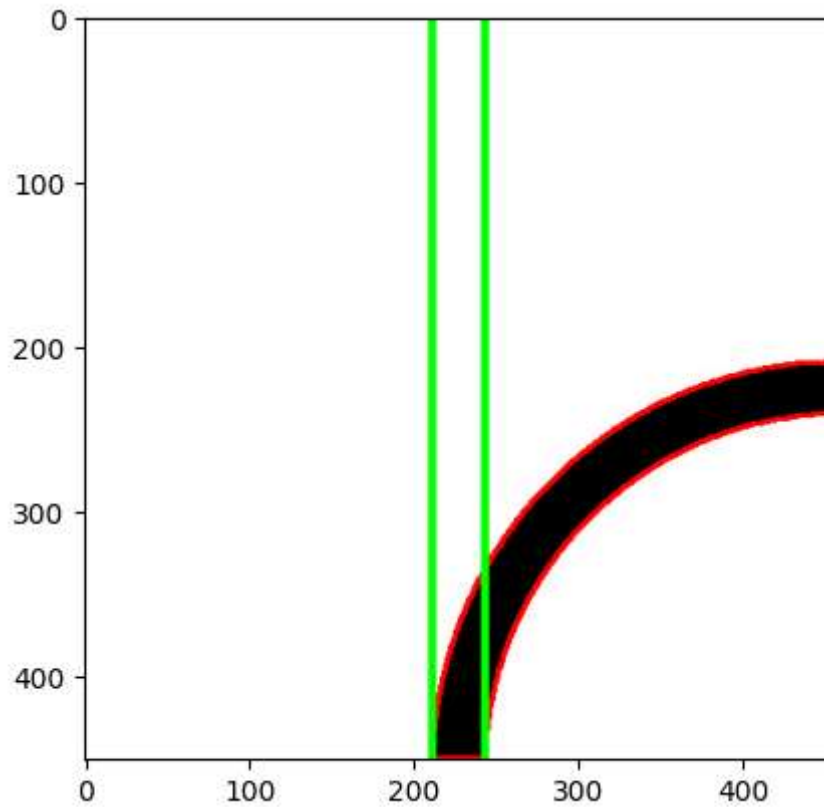


```
In [145... img = cv2.imread('datas/ladrinho2_direita.jpg')  
erro = detect_line(img)  
plt.imshow(img);  
print('The error in degrees:', erro)
```

0 211.0

Right

The error in degrees: 44



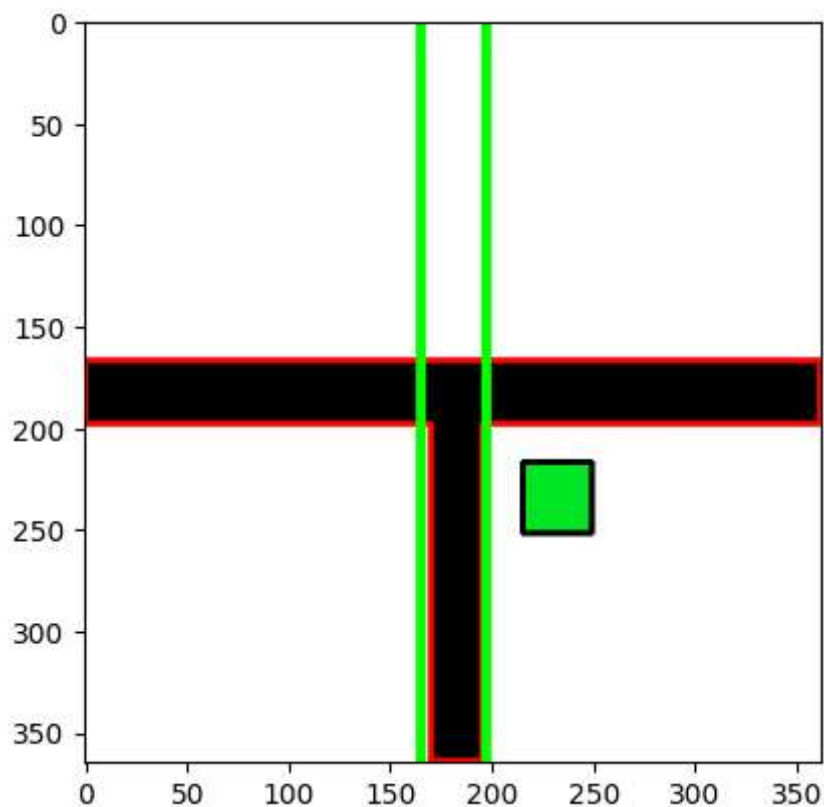
```
In [146... img = cv2.imread('datas/ladrilho_verde.jpg')
erro = detect_line(img)
plt.imshow(img);
print('The error in degrees:', erro)
```

0 0.0

Green square

Has the colour green on the image

The error in degrees: -40

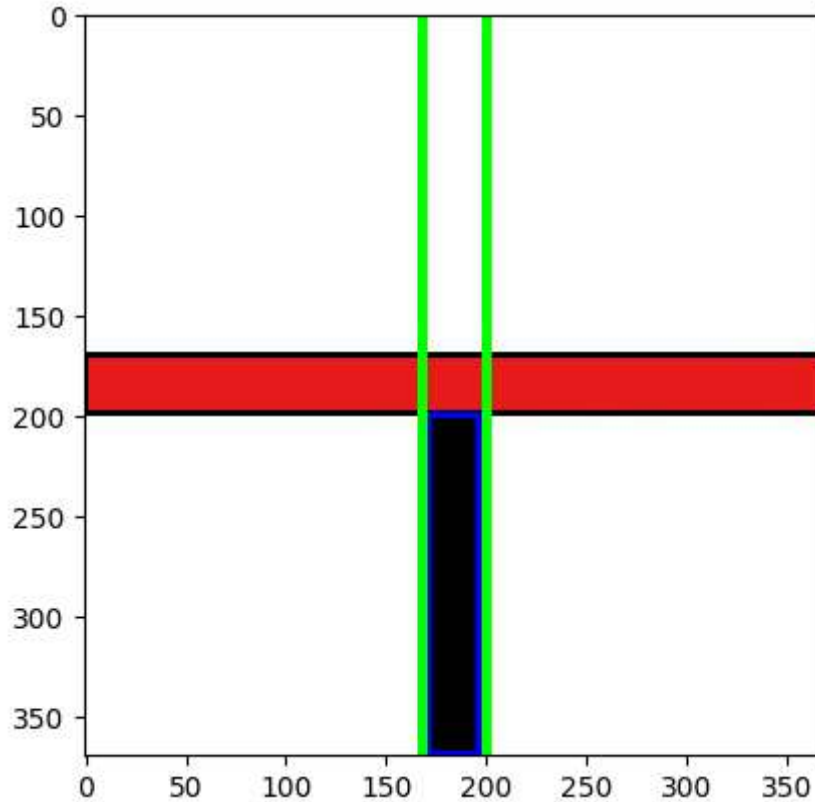


```
In [147... img = cv2.imread('datas/ladrilho_vermelho.jpg')
erro = detect_line(img)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));
print('The error in degrees:', erro)
```

No curve

Has the colour red on the image

The error in degrees: 0

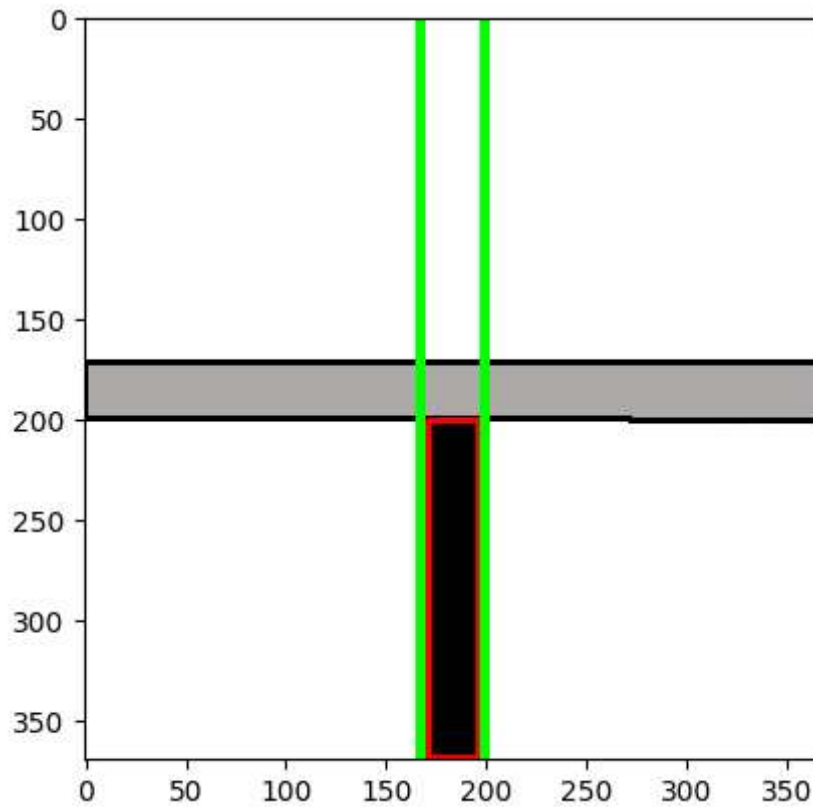


```
In [148... img = cv2.imread('datas/ladrilho_cinza.jpg')
erro = detect_line(img)
plt.imshow(img);
print('The error in degrees:', erro)
```

No curve

Has the colour gray on the image

The error in degrees: 0



```
In [149... img = cv2.imread('datas/ladrinho6.jpg')
erro = detect_line(img)
plt.imshow(img);
print('The error in degrees:', erro)
```

0 0.0

Green square

The error in degrees: -40

